

aprintc: Self-Improving Agents & Robots via Outcome-Anchored Apprenticeship Distillation

Ankur Srivastava

TECHNICAL REPORT · JUNE 2026

ABSTRACT. Production AI agents are stateless: each deployment is frozen, and improving one means a human manually rewriting prompts or curating datasets. We present **aprintc**, a framework that lets a deployed agent improve *itself* from its own real-world behavior, under human oversight. aprintc treats a live **parent** agent's stream of real interactions as an experiential substrate — analogous to how a world-foundational model learns a usable model of an environment from observation — and distills from it an improved **child** agent. Crucially, the child is *not a separate model*: parent and child share one frozen engine (a closed-weight model — language, vision, or multimodal — together with its retrieval and tools), and differ only in a **Playbook** — a compact, versioned, non-parametric policy artifact (system prompt, directives, exemplars, and avoidance lessons) distilled from the parent's outcome-labelled trajectories. Promotion is therefore a reversible configuration swap rather than a weight update, which makes the method applicable to *closed-source* models that cannot be fine-tuned. Quality is **anchored on task outcomes** and fused with explicit feedback and a *recused* LLM judge (judge $\neq$ policy) whose trust is learned from its historical agreement with ground truth. A child is promoted only after passing a statistical acceptance gate (win-rate, Wilson confidence interval, bounded downside) and zero-tolerance regression and safety gates. We instrument arbitrary agents non-invasively at four stable protocol boundaries — the model API, the tool/MCP layer, telemetry (OpenTelemetry), and the agent (A2A) layer. The framework is **modality- and embodiment-agnostic**: the engine may be a language, vision, video, or multimodal model, or a robot's real-time perception–reasoning–control stack in which a *parent robot* teaches an apprentice from real sensorimotor outcomes — though the empirical study here is in language domains. On a deliberately under-specified support agent with genuine headroom, distilled children win **65–85%** of head-to-head comparisons against their parent and clear the gate; on a parent already near its capability ceiling, win-rate collapses to chance and the gate correctly *rejects* — the central, honest finding of this work: **playbook distillation reliably extracts latent competence, but cannot manufacture capability the base model lacks.**

1. Introduction

The dominant limitation of AI agents in production is not raw capability — it is that they **do not learn from experience**. A support assistant that mishandles the same refund edge case on Monday will mishandle it again on Friday; a warehouse robot that fumbles the same awkwardly-shaped item gains nothing from the attempt. Today, closing that gap is a manual loop: an engineer reads transcripts, infers what went wrong, edits a prompt or assembles a fine-tuning set, redeploys, and hopes. This loop is slow, unattributable, and does not scale across a fleet of agents.

aprntc asks a different question: *can a deployed agent improve itself from its own behavior, continuously and safely, with a human only at the final gate?* Our answer reframes self-improvement as **apprenticeship**. A live **parent** agent serves real users. An **apprentice** (“child”) shadows it, distills lessons from what the parent did well and badly, becomes a measurably better successor, and — whether **auto-promoted** for low-risk changes or after **human approval** — replaces the parent. The cycle then repeats from the new baseline: *observe* → *label* → *distill* → *evaluate* → *promote* (Figure 1).

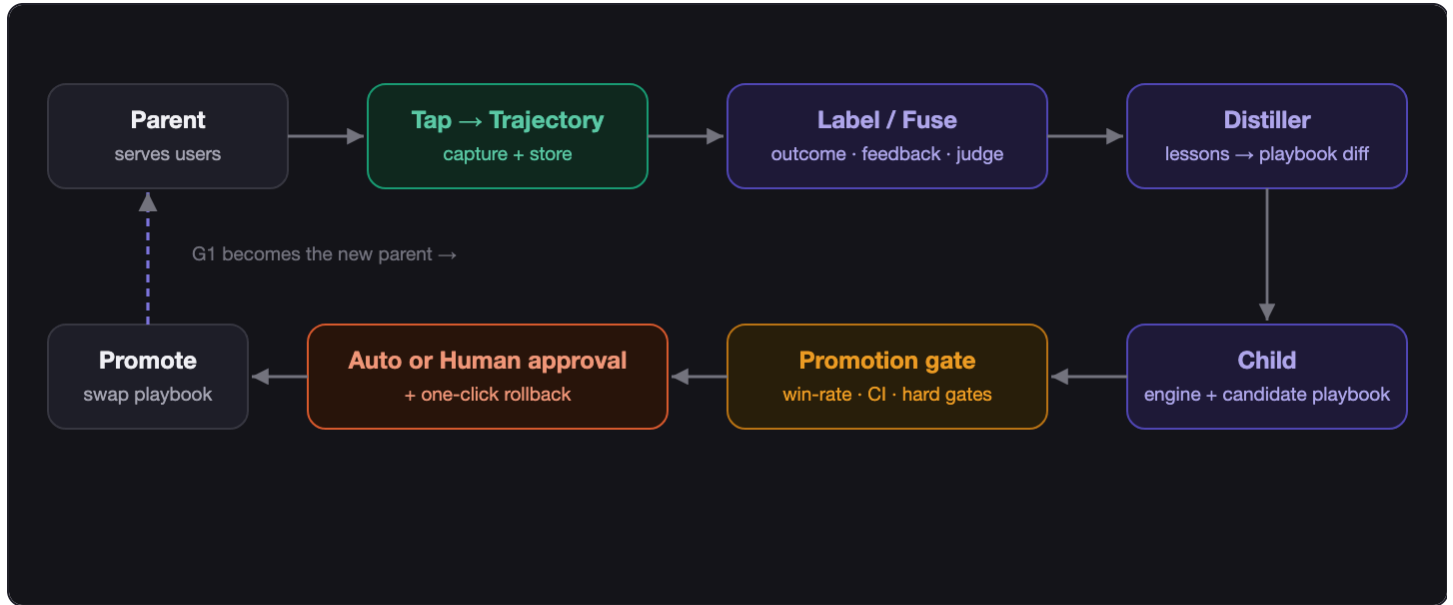


Figure 1. The apprenticeship loop. The parent serves users; a read-only tap stores outcome-labelled trajectories; the distiller mines a candidate Playbook; the child (same engine, new Playbook) is judged against the parent at a statistical gate; on approval it is promoted — automatically for low-risk diffs or by a human — and becomes the new parent. The dashed arrow is the only write back to the live agent.

A world-foundational-model framing. Recent progress in world models learns a usable, predictive model of an environment purely from observation, then uses it to plan or to generate better behavior [16, 17, 18]. aprntc adopts the same posture, but the “environment” is the parent agent acting in the *real world* — real users, real tools, real outcomes — and the artifact we learn is not a simulator of dynamics but an **improved policy** grounded in those observed dynamics. The parent’s logged rollouts are the training distribution; the child is the improved policy distilled from them. We do not assume a simulator, a reward oracle, or the ability to update model weights. We assume only that we can *observe* the agent, *score* its outcomes, and *re-prompt* it.

The closed-weight constraint forces a non-parametric policy. The strongest models available to most builders are closed-source and cannot be fine-tuned. We therefore make the child’s policy a **Playbook**: a structured, human-readable, version-controlled bundle of a system prompt, imperative directives, situation→response exemplars, and “watch-out” avoidance lessons. The child is the *same engine* running a *different Playbook* (Figure 2). This single design choice yields three properties a weight update cannot match: every behavioral change is **attributable** to the trajectories that produced it; every promotion is **reversible** with one configuration swap; and the method is **model-agnostic**, working on closed and open models alike.

Any model, any modality, any embodiment. Nothing in this loop is specific to language. aprntc treats the engine as a black box it can only *observe*, *score*, and *re-prompt*, so the same machinery applies wherever an agent produces scorable behavior — a vision or video model, a multimodal model, or a *composite* pipeline of several. It extends, in particular, to **robotics**: a robot is itself a real-time multi-agent system — perception models (vision, depth, audio) build a world state, a planner or language model reasons over it, and a control policy acts — and a

deployed *parent robot* generates exactly the experiential substrate an apprentice can learn from. There, trajectories are sensorimotor episodes, the outcome is task success and safety, and the Playbook generalizes to whatever in-context control surface the stack exposes: planner prompts, a reusable skill/behavior library, retrieved demonstrations, and tuned guardrails. We develop this generalization in §4.9; our experiments (§5–6), however, are deliberately confined to language domains where outcomes are crisp and reproducible.

Contributions.

1. **An apprenticeship formulation** of agent self-improvement (§3): offline policy improvement from a parent's logged, outcome-labelled trajectories, restricted to a non-parametric (in-context) policy class.
2. **Outcome-anchored quality estimation** (§4.2): a fusion of deterministic outcome signals, explicit feedback, and a *recused* LLM judge, with fusion weights learned from each source's agreement with ground truth — a reward model that resists self-preference and reward hacking.
3. **Attributable playbook distillation** (§4.4): a procedure converting labelled successes and failures into an auditable Playbook *diff*, with provenance linking each lesson to its source episodes.
4. **A statistical promotion gate** (§4.6): a child ships only on a frozen held-out set it never saw, gated on win-rate, a Wilson confidence bound, bounded loss-rate, and zero-tolerance regression and safety checks, with one-click rollback and a lineage DAG.
5. **Non-invasive, framework-agnostic capture** (§4.1): a tap instrumenting arbitrary agents at four stable protocol boundaries (model, tool/MCP, telemetry/OTel, agent/A2A), normalized into one canonical trajectory schema.
6. **An honest empirical characterization** (§6): distillation produces large, gate-clearing gains *only* where the parent has exploitable headroom and ground truth exists; against a saturated parent it correctly yields no significant lift.

2. Related Work

Apprenticeship and imitation learning. Learning a policy by observing an expert is classical [1, 2]. Behavioral cloning and DAGger [2] imitate expert *actions*; inverse RL [1] recovers the expert's *reward*. aprntc differs on two axes: the “expert” (the parent) is *not* assumed optimal — it is the thing we aim to surpass — and we improve over it by distilling *outcome-discriminated* lessons rather than cloning behavior wholesale.

Offline reinforcement learning. Improving a policy from a fixed log without further environment access is the offline-RL setting [3]. We inherit its central tension — distribution shift between the logged and learned policies — but operate in a non-parametric, in-context policy class and substitute a heavyweight value function with a calibrated, outcome-anchored reward estimate and an explicit, statistically-gated deployment.

Learning from human and AI feedback. RLHF [4, 5] and RLAIIF / Constitutional AI [6] align models with preference signals. Our recused judge is an RLAIIF-style reward model, but we treat it as a *prior, never the authority*: it is outranked by deterministic outcomes and explicit user feedback, and its weight is *learned* from how often it has historically agreed with them. This directly targets the self-preference bias of LLM judges [9].

Memory-augmented and self-improving agents. Reflexion [10] converts failures into verbal self-feedback; Voyager [11] accumulates a reusable skill library; Generative Agents [12] maintain reflective memory; STaR and expert iteration [13, 14, 15] bootstrap a model from its own successful rollouts. aprntc systematizes this family into a *productized, gated, multi-tenant loop*: persistent outcome-labelled trajectories, a curated and version-controlled experience memory, an auditable distillation step, and — the piece these works omit — a **statistical, human-gated promotion** with regression and safety guarantees and instant rollback.

World models. World-model and world-foundational approaches [16, 17, 18] learn environment dynamics from observation to enable planning or generation. We borrow the *learn-from-observation-of-the-real-world* stance but target an improved control policy grounded in observed outcomes rather than a learned dynamics model.

Robot learning and learning from demonstration. Learning control policies from example trajectories is a mature field [23]: behavioral cloning and DAGger [2] in the small, and at scale a new generation of **vision-language-action (VLA)** models — RT-2 [24], the cross-embodiment RT-X models trained on the Open X-Embodiment dataset [25], OpenVLA [26], diffusion policies [27], and flow-based policies such as π_0 [28] — that map pixels and instructions directly to motor actions. aprntc is *complementary* to this line rather than a competitor: such a policy is precisely the kind of frozen **engine** a parent robot runs, and aprntc wraps it in an outcome-gated apprenticeship loop — distilling a reversible, attributable policy artifact from the parent robot’s real rollouts and adding the **statistical, safety-gated promotion** these works leave to manual retraining. Cross-embodiment transfer [25] is, in turn, the embodied analogue of our fleet-level lesson sharing (§7).

Agent interoperability and observability. We build capture on open standards — OpenTelemetry GenAI conventions, the Model Context Protocol (MCP) [20], and the Agent-to-Agent (A2A) protocol [21] — and wrap mature open-source gateways rather than reimplementing transports. We are explicit (§4.1) that aprntc is *not* an observability product: capture is best-effort and quality-signal-oriented, not exhaustive tracing.

Parameter-efficient fine-tuning. LoRA and related PEFT methods [19] make weight updates cheap *when weights are accessible*. We position parametric internalization as **Phase 2** (§4.8): once a corpus of gate-validated lessons accumulates, the same supervision that built Playbooks can train an adapter (“agent SFT”), compressing in-context lessons into weights when prompt length becomes the binding constraint.

3. Problem Formulation

Let the parent agent be a policy π_P mapping a task context x (user query, retrieved context, tool state, history) to a trajectory τ and a final response y . Deployed over a user population, π_P induces a distribution of trajectories $\mathcal{D}_P = \{(x_i, \tau_i, y_i)\}$. Each trajectory receives an **outcome** $o_i \in [0,1]$ from a fusion of signals (§4.2): task resolution, explicit user feedback, implicit acceptance, and a recused judge, ordered by reliability. We write x , τ , y in text for concreteness, but they are modality-neutral: x and τ may carry images, video, or sensor observations, and y may be an action sequence rather than a message; the formulation requires only that outcomes be *scorable*.

Objective. Find an improved policy π_C maximizing expected outcome,

$$\pi_C = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mathcal{P}(x)} [o(\pi(x))],$$

subject to (i) **no environment access**: candidates may only be re-run offline on logged or held-out tasks; (ii) **a frozen engine**: base model, retrieval, and tools are fixed; and (iii) **a safety/regression floor**: π_C must not regress a fixed suite of previously-correct or safety-critical cases.

The policy class. Because the engine is frozen and weights are inaccessible, Π is the set of behaviors reachable by varying the **Playbook** θ — the in-context policy parameters — with the engine held constant: $\pi_\theta(x) = \text{Engine}(x; \theta)$. A “generation” G_n is one Playbook θ_n ; G_0 is the parent’s. Self-improvement is iterated policy improvement over generations, $\theta_0 \rightarrow \theta_1 \rightarrow \dots$, where each step is a distillation operator $\theta_{n+1} = \mathcal{T}(\theta_n, \mathcal{D}_{G_n})$ followed by an acceptance test. This is policy iteration with a non-parametric actor, a learned outcome-anchored critic, and a *statistical* improvement check standing in for a monotonic-improvement guarantee.

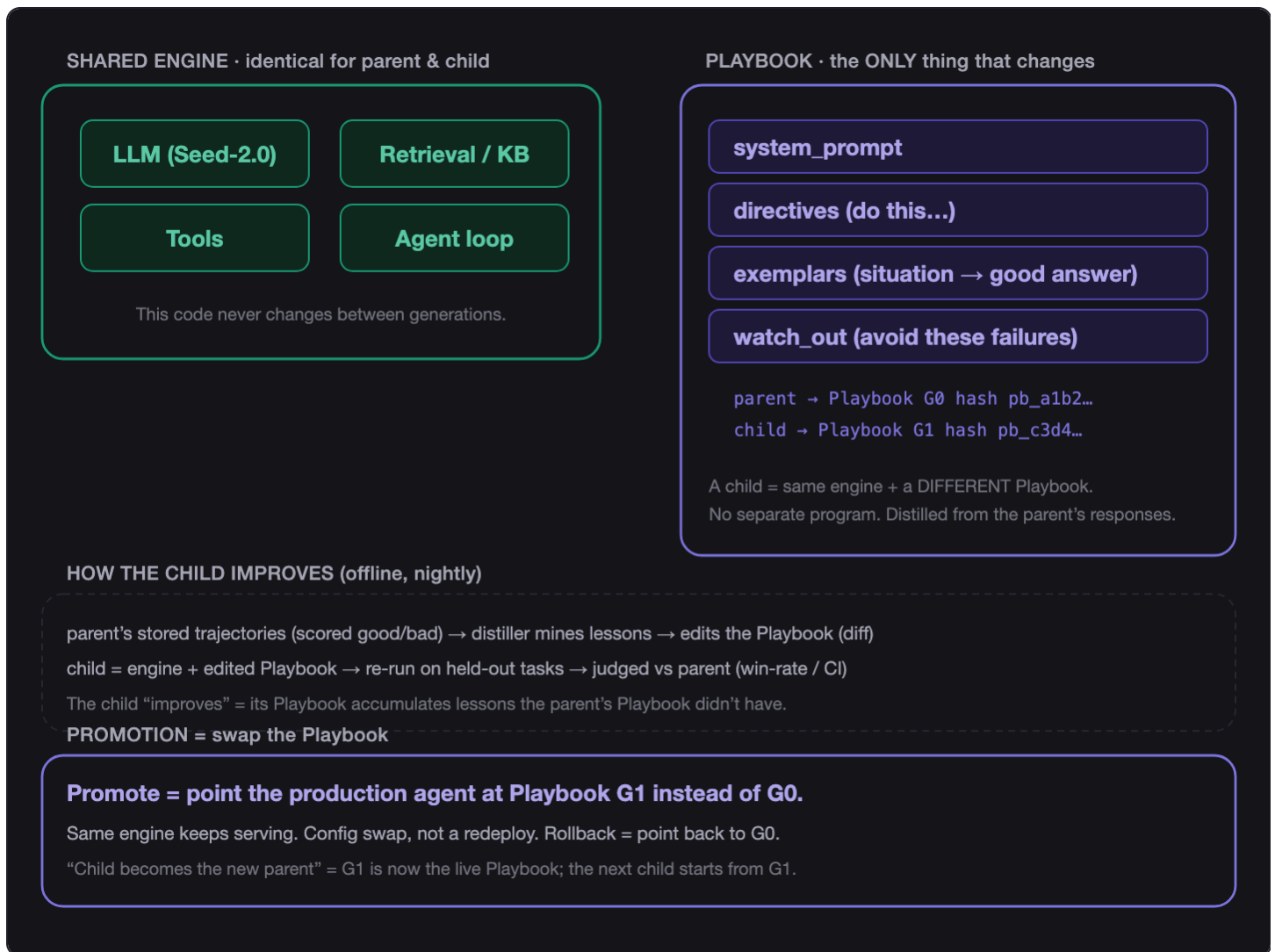


Figure 2. The child is not a separate program. Parent and child share one frozen engine (LLM, retrieval, tools, agent loop); they differ only in the *Playbook* — system prompt, directives, exemplars, and watch-out lessons. The child improves because its Playbook accumulates lessons the parent's did not have; promotion swaps the Playbook (a reversible configuration change, not a redeploy).

4. The aprntc System

aprntc runs **beside** production, never in the request path; if it is down or wrong, the parent is unaffected (Figure 3). It observes the parent through a read-only tap (inbound) and serves the active Playbook the agent pulls (outbound). The only write back to production is a **gated configuration swap**.

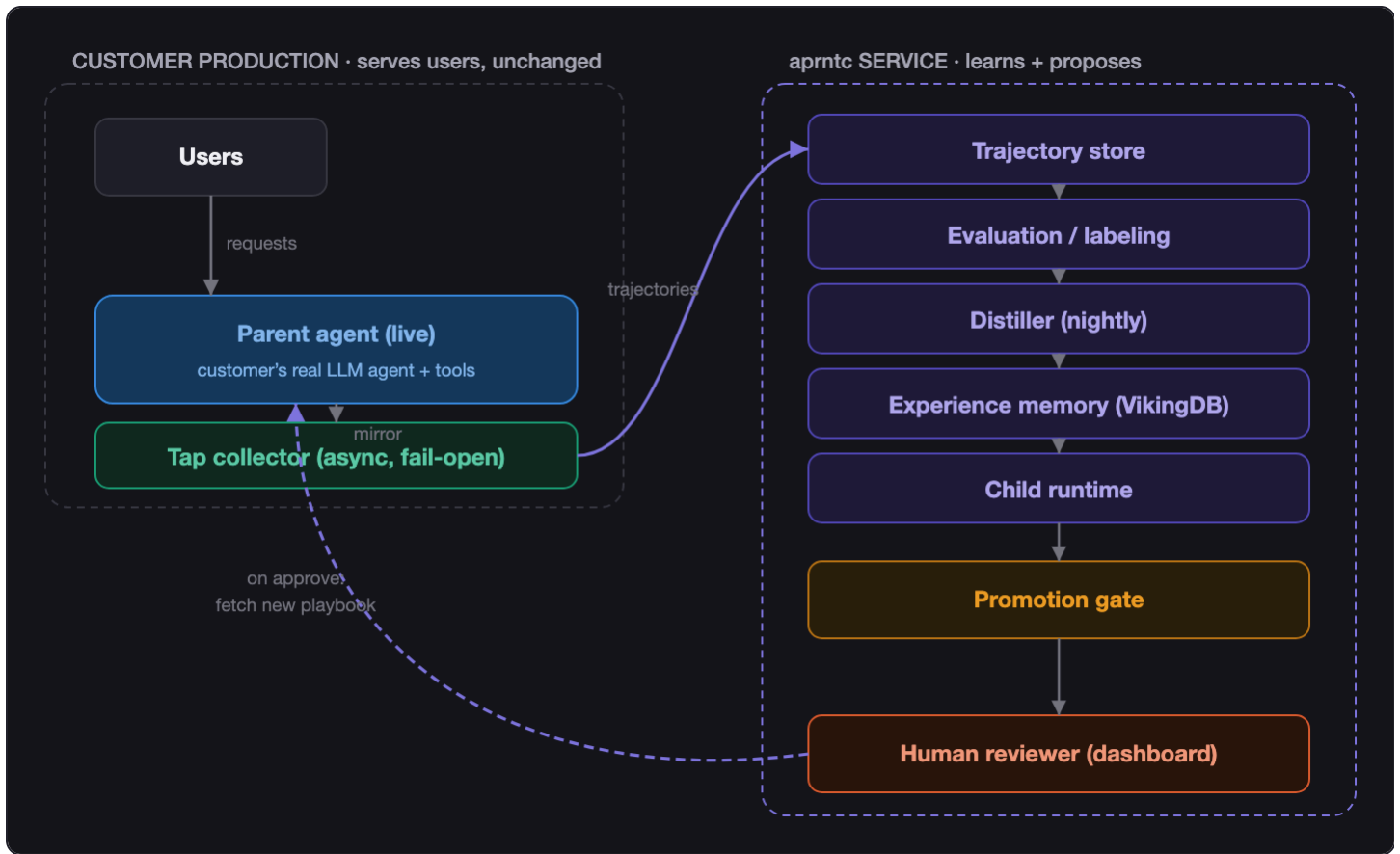


Figure 3. System architecture. Customer production (left) serves users unchanged; a fail-open tap mirrors interactions to the aprntc service (right), which stores trajectories, labels them, distills, retrieves from experience memory, runs the child, and gates promotion behind a human reviewer. On approval, the parent fetches the new Playbook — the sole, reversible write back.

4.1 Observation: the tap and the canonical trajectory

To learn from *any* agent — not just ones we wrote — we intercept at **stable protocol boundaries rather than framework internals** (Figure 4). Four boundaries are instrumented by five collectors:

- **Model boundary — SDK wrapper.** Wraps the model client of agents we own; full-fidelity tool capture (name, arguments, result, timing, errors).
- **Model boundary — egress proxy (LiteLLM).** The agent points its `base_url` at our OpenAI-compatible proxy; captures model traffic for *any* agent including closed-source. Tool calls are reconstructed (partial) from the function-calling loop.
- **Tool boundary — MCP gateway → OTel.** For agents whose tools are MCP servers; direct, full tool capture (MCP tools only).
- **Telemetry boundary — OpenTelemetry ingester.** For OTel-instrumented frameworks (LangChain, LlamaIndex, CrewAI); fidelity conditional on the spans the source emits.
- **Agent boundary — A2A interceptor.** For multi-agent systems communicating over the Agent-to-Agent protocol; task/artifact-grained capture of delegated work and returned artifacts.

Every collector normalizes into **one canonical Trajectory** (Episode → Turn → Step, with multimodal parts — text, images, audio, or sensor frames — reasoning traces, tool args/results, and outcome labels), and the tap is **asynchronous and fail-open** — it never slows or breaks the parent. Tool-call fidelity *degrades gracefully* by how the target was built (SDK wrapper → MCP → OTel → egress proxy → A2A); this tiering is a property of the boundary-based design, not a gap. By scoping principle, capture targets **episode-level quality signal**, not

exhaustive tracing: aprntc is a *learning* product, not observability. PII is scrubbed at ingest and raw trajectories carry a retention TTL; lessons are derived from scrubbed text only.

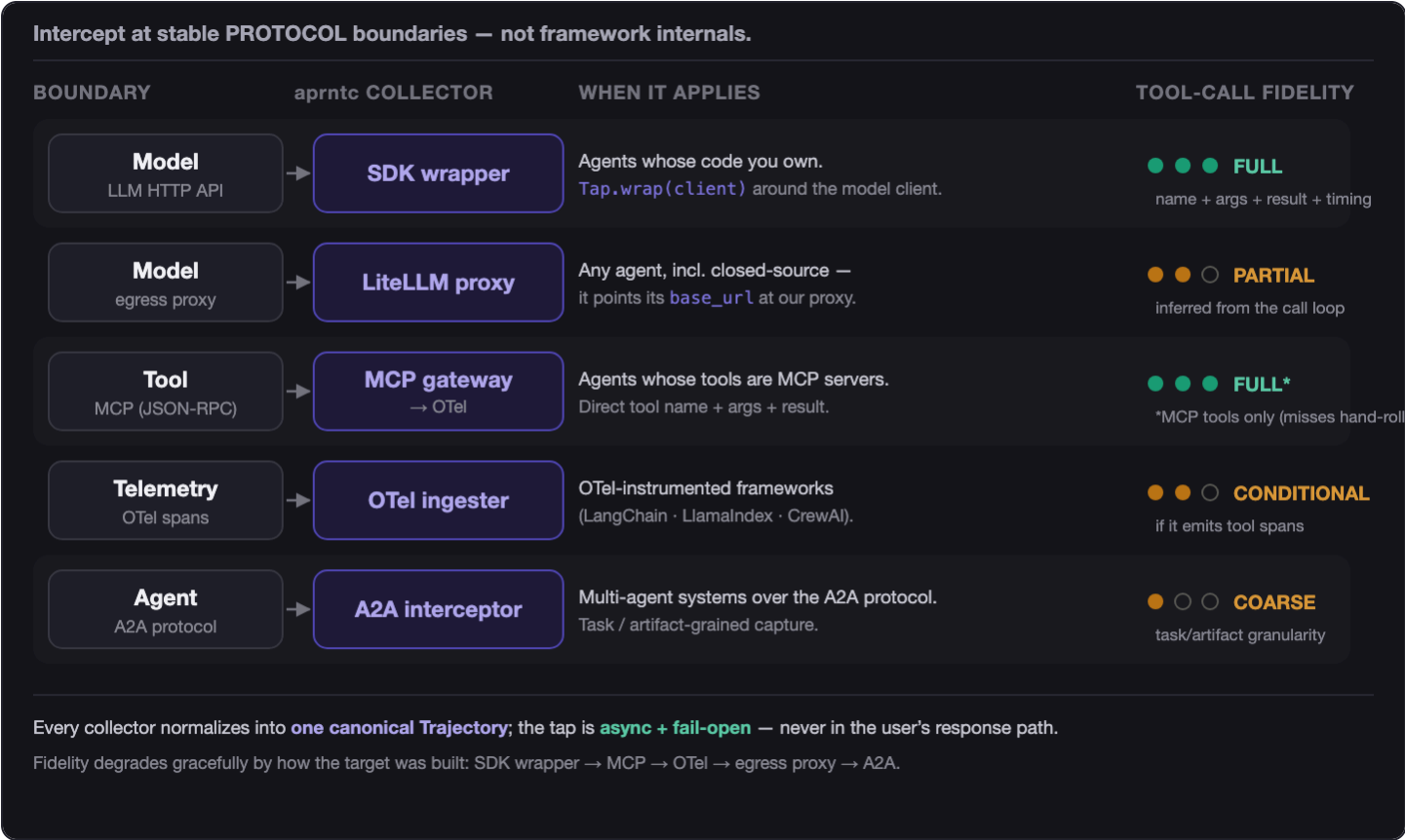


Figure 4. The four tap boundaries and five collectors, the scenario each fits, and its tool-call fidelity. Fidelity degrades gracefully by how the target agent was built; all four boundaries, including the agent-to-agent (A2A) layer, are instrumented.

4.2 Outcome-anchored quality labeling

A learned policy is only as good as the reward it optimizes. We estimate per-trajectory quality from multiple sources, **ordered by reliability**: outcome > explicit feedback > implicit signals > LLM judge.

- **Outcome** (the anchor): deterministic, task-specific — resolved-within-session for support; groundedness / citation-correctness for retrieval QA.
- **Explicit feedback**: user thumbs up/down.
- **Implicit**: no-rephrase, answer copied, session satisfied.
- **Recused judge**: a pairwise LLM judge that is *a different model from the policy* (judge ≠ policy) to neutralize self-preference bias [9]; a *prior*, never the authority.

The signals are fused into a calibrated reward; **fusion weights are learned** from each source's historical agreement with the anchor, so a judge that disagrees with real outcomes is automatically down-weighted. This is our principal defense against **reward hacking**: a candidate that games the judge but does not move real outcomes is detected because judge-vs-outcome divergence collapses the judge's weight. A separate **gold set** (frozen reference Q&A with citations) is reserved for the gate and for periodic judge calibration and is *never* visible to distillation.

4.3 Experience memory

Validated lessons are written to a curated, generation-versioned **experience memory** (a vector index; VikingDB in our reference deployment) — not a document corpus, but a deliberately small index of *what worked and what to avoid*, each entry tagged with generation, reward, lesson type, and PII status. Retrieval is filtered hybrid search with MMR de-duplication, so the child is injected with a handful of *relevant, diverse, high-reward, generation-appropriate* exemplars and avoidance notes under a token budget.

4.4 Apprenticeship distillation: building the Playbook

The distiller is the policy-improvement operator $\mathcal{T}$. Nightly, it buckets the parent's labelled trajectories into successes and failures and mines them into Playbook edits: **successes** → **directives and exemplars** (recurring patterns behind good outcomes become imperative rules and situation→response exemplars); **failures** → **watch-outs** (recurring patterns behind bad outcomes become explicit avoidance lessons). Critically, the distiller emits an **attributable diff** — it *edits* the parent Playbook rather than regenerating it — and every added line carries **provenance** (the episode IDs that justify it). A reviewer can read exactly what changed and why. The result is candidate generation G_{n+1} : the same engine, the parent's Playbook plus a small, auditable set of distilled lessons.

4.5 The child runtime

The child is the parent's orchestration with θ_{n+1} installed: at inference it retrieves relevant lessons from experience memory and runs them through the identical engine. There is no separate model and no separate codebase — “child” denotes a Playbook generation, not a program. “The child becomes the new parent” means G_{n+1} is now the live Playbook and the next apprentice starts from it.

4.6 Promotion: the statistical gate, lineage, and rollback

Distillation proposes; the gate disposes. A candidate is evaluated by **offline counterfactual replay** on a **frozen held-out set the distiller never saw** (so the child cannot “study for the test”). Parent and child run the *same* tasks; the recused judge compares each pair. A candidate is promotable only when **all** of the following hold:

Check	Threshold	Type
Win-rate (child vs parent)	$\geq 55\%$	quality
Confidence	95% Wilson CI lower bound $> 50\%$	statistical
Loss-rate	$< 10\%$	bounded downside
Regression suite	0 failures	hard gate
Safety probes	0 new violations	hard gate
Promote + rollback	both work from the UI	operational

Table 1. The promotion gate. All checks must pass on a frozen held-out set the distiller never saw.

The **Wilson score interval** [8] forces both a real margin *and* a powered sample size — a child cannot pass on five lucky examples. The regression and safety gates are zero-tolerance: a 60%-better child with a single new safety violation is rejected outright. Each promotion is recorded in a **lineage DAG**; **rollback** is a one-click revert to the prior Playbook. Low-risk diffs may be **auto-promoted** under a default-off, hard-gated policy; higher-risk diffs require human approval. Auto-promotion is *earned* only after the judge ↔ outcome trust and the human-gated process have proven reliable.

4.7 Online: shadow and canary

Beyond offline replay, the child can run as a **shadow** on live traffic (its outputs discarded but judged against the parent's), and post-promotion rollout can be a **canary** with automatic rollback if a live metric degrades. These give production-grounded evidence without ever putting an unproven child in front of a user.

4.8 From non-parametric to parametric: agent SFT (Phase 2)

Playbook distillation is **Phase 1** — it changes *how the engine is prompted*, not the weights. It is the right first move precisely because most value is reachable *without* training infrastructure, and because a bad Playbook diff is one row to revert whereas a bad fine-tune is opaque and risky. But two forces motivate a parametric **Phase 2**: prompt length and latency rise as lessons accumulate, and some lessons are better internalized than retrieved. In Phase 2 the *same* gate-validated supervision that built Playbooks becomes a training set for an **agent SFT / PEFT** step (e.g., a LoRA adapter [19]) when model weights are accessible, compressing in-context lessons into parameters once the prompt budget becomes the binding constraint. Phase 2 *compresses*; it never replaces the memory-and-playbook layer, which remains the attributable, reversible control surface.

4.9 Generality: any model, any modality, and robotic apprentices

The loop of §4.1–4.8 makes no assumption that the engine is a language model. It needs only three operations — **observe** a trajectory, **score** its outcome, and **re-prompt** the engine with a new Playbook — so it instantiates wherever an agent produces behavior that can be scored.

Multimodal engines. The canonical trajectory already carries multimodal parts (§4.1), and the recused judge can itself be multimodal, so vision, video, audio, and mixed-modality agents fit unchanged. The only domain-specific element is the outcome anchor (§4.2): a grounded-answer check for document VQA, a detection or segmentation metric for vision, a task-completion signal for tool-using agents. Everything downstream — fusion, memory, distillation, the gate — is identical.

Robotic apprentices. A robot is not a single model but a real-time **stack of agents**: perception models (vision, depth, audio, proprioception) construct a world state, a planner or language model decides what to do, and a control policy — increasingly a **vision-language-action** model [24, 26, 28] or a diffusion policy [27] — emits motor actions. These components communicate continuously, frequently across exactly the agent-to-agent boundary our A2A collector taps (§4.1). A deployed **parent robot** therefore emits a rich experiential substrate of *sensorimotor episodes*, and the apprentice distills a better successor from it. The mapping is direct:

- **Trajectory** = a sensorimotor episode (observations → actions → result), captured at the perception, planner, and A2A boundaries.
- **Outcome** = physical task success and safety (grasp succeeded, object placed, no collision, within force/time limits) — often *more* objective than a text rubric, and frequently auto-labelled by the environment.
- **Playbook** = the in-context control surface of the stack: planner/system prompts, a reusable **skill or behavior library** (in the spirit of Voyager [11]), retrieved exemplar demonstrations, and tuned guardrails — all reversible without retraining the underlying policy.
- **Promotion gate** = offline replay *in simulation*, then a **teleoperated or shadow** trial, before any physical canary; the zero-tolerance safety gate (§8) is non-negotiable when actions move mass in the real world.

Phase boundary, restated for control. Phase 1 (§4.4) tunes the *non-parametric* surface above. Phase 2 (§4.8) is the parametric step: the same gate-validated episodes become a fine-tuning set for the VLA / control policy itself (e.g., a LoRA adapter [19] over an OpenVLA- or π O-class model [26, 28]) when its weights are accessible, compressing distilled skill into the policy once the prompt/skill budget binds.

We stress that this section describes the framework's generality **by construction**; the empirical results in §6 are obtained in language domains, and embodied validation — physical-task outcomes and sim-to-real gating — is future work (§9).

5. Experimental Setup

We instantiate the framework (§4.9) in the **language setting**, where outcomes are crisp and runs are reproducible; the multimodal and robotic generalizations are not evaluated here.

Models. Reference deployment on BytePlus ModelArk: policy/agent = Seed-2.0-pro; recused judge = DeepSeek-V4-pro (judge $\neq$ policy). Experience memory = VikingDB. All capture, distillation, and gating run against live infrastructure, not stubs.

Tasks. Two synthetic, PII-free domains: (a) a **customer-support** assistant (outcome = resolved-within-session; minutes-latency outcome joining + thumbs feedback), and (b) a **retrieval-QA** assistant over a public/synthetic corpus with a 50–100 item reference gold set (outcome = groundedness / citation-correctness + acceptance). Demo data is synthetic and reproducible by design.

Protocol. For each domain we register the parent's current system prompt as G_o , capture trajectories, label them, distill G_I , and run the gate of §4.6 on a frozen held-out split. We vary the **parent's headroom** — a deliberately thin, under-specified parent vs. a strong, well-specified one — to isolate when distillation helps.

Metrics. Pairwise win-rate (recused judge), Wilson 95% CI, loss-rate, regression and safety pass/fail; plus operational checks (promote + rollback from the dashboard).

6. Results

Distillation produces large, gate-clearing gains where the parent has headroom. On the deliberately under-specified support parent — one with recurring, fixable failure modes and available ground truth — distilled children win **65–85%** of head-to-head comparisons against their parent, with the Wilson lower bound clearing 50% and loss-rate within bound. These candidates pass the full gate, including zero regressions and zero new safety violations, and were promoted and rolled back from the dashboard (Figure 5, right).

Against a saturated parent, the method correctly finds nothing. When the parent is already near the base model's ceiling for the task, the same pipeline yields a child whose win-rate sits at roughly **50–52%**, with a confidence interval straddling chance. The gate **rejects** it (Figure 5, left). This is not a failure of the system but its **integrity**: aprntc declines to ship noise.

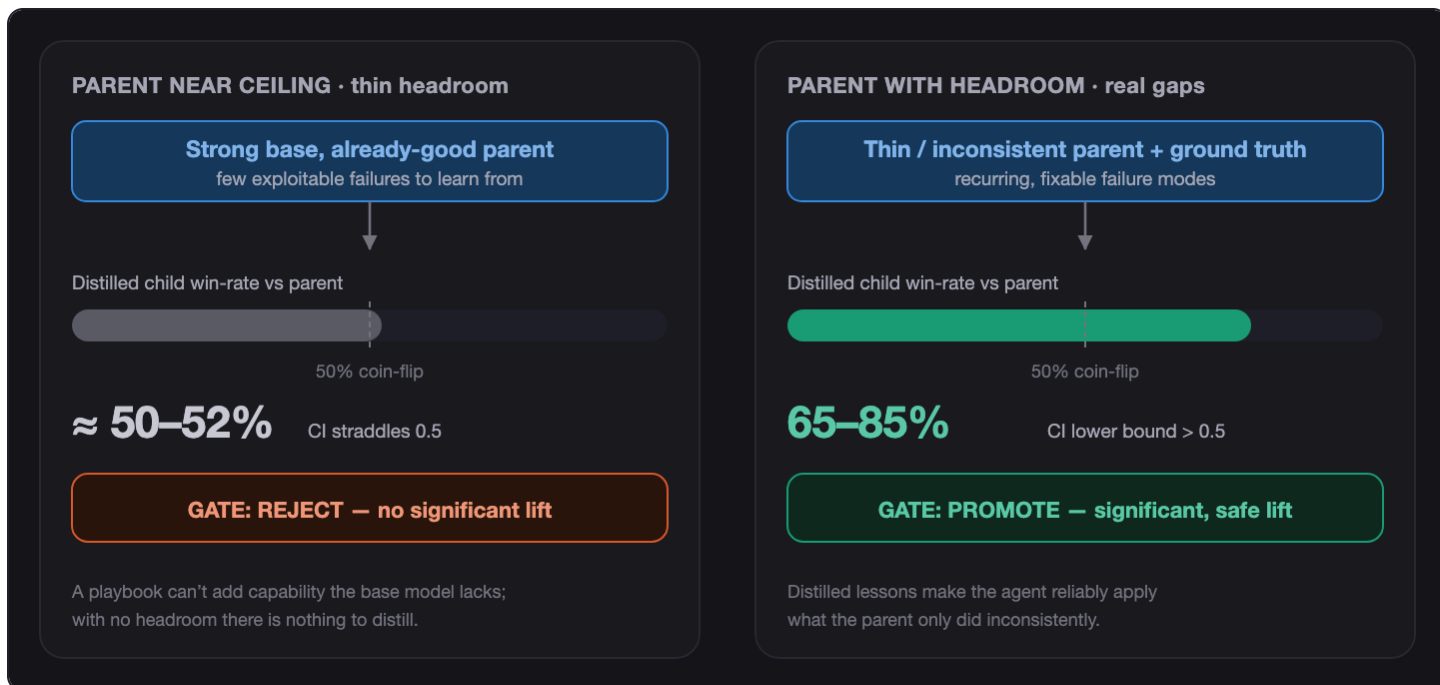


Figure 5. The headroom finding. (Left) Against a parent already near its capability ceiling, the distilled child's win-rate sits at chance and the gate rejects. (Right) Against a parent with genuine, recurring headroom and a reliable outcome signal, the child wins 65–85% and clears the gate. Playbook distillation extracts latent competence; it does not manufacture new capability.

The headroom finding. Taken together (Figure 5), these define when playbook distillation works: it requires (i) a parent that *under-performs its base model's latent capability* on recurring patterns, and (ii) a reliable outcome signal to steer toward. A Playbook makes the engine **reliably apply** competence it already possesses; it cannot **add** capability the base model lacks. We consider this the most important and most honest result of the work, and we report it rather than tuning the demo to manufacture a uniform “pass.”

Ablations (qualitative). Removing experience-memory injection degrades the child toward the parent (the lessons are doing the work). Replacing the recused judge with a self-judge (judge = policy) inflates apparent win-rate without a corresponding outcome gain — the self-preference signature [9] — which the learned fusion weights suppress when outcomes are present. Removing the held-out discipline (letting distillation see the gold set) inflates offline win-rate while real-outcome gains vanish — textbook reward hacking, which is exactly what the frozen gold set and outcome anchor exist to prevent.

7. Analysis & Discussion

Why apprenticeship, not cloning. We never imitate the parent wholesale; we imitate its *outcome-discriminated* behavior — amplifying what scored well, suppressing what scored badly. This is what lets the child *exceed* a non-optimal parent rather than reproduce it, and it is why the quality signal (not the behavior log) is the scarce resource.

Why non-parametric first. The Playbook is a control surface a human can *read, diff, and revert*. For a self-modifying system that will eventually act with reduced supervision, that auditability is not a convenience — it is the safety substrate. Weight updates trade it away for capacity we do not yet need.

The world-model lens, precisely. Like world-foundational models, aprntc learns from *observation of the real world* and uses what it learns to *act better*. Unlike them, it learns a policy improvement rather than a dynamics model, and it is grounded by real outcomes rather than self-generated rollouts — closer in spirit to expert iteration [14] than to model-based planning, but with the “expert” being the system's own gated, outcome-validated successor.

Fleets and transfer. Because lessons are explicit artifacts, they transfer: one apprentice can supervise many parents, and gate-validated lessons can be shared across agents (with per-tenant isolation). This turns single-agent self-improvement into fleet-level knowledge accumulation. The same property carries to embodied fleets — gate-validated skills can transfer across heterogeneous robots, the practical mirror of cross-embodiment learning [25].

8. Safety and Broader Impacts

A self-modifying agent demands a conservative safety case. aprntc's defenses are structural:

- **Human-gated by default; auto-promotion is earned.** Every promotion ends in mandatory approval unless a diff is low-risk *and* the auto-policy is explicitly enabled after trust is established.
- **Zero-tolerance regression and safety gates.** Previously-fixed failures must stay fixed (no catastrophic forgetting); red-team probes must show no new violation.
- **Reversibility.** Promotion is a configuration swap; rollback is one click; lineage is fully recorded.
- **Reward-hacking resistance.** Outcome anchoring, the recused judge, learned fusion weights, and a frozen gold set jointly ensure that gaming the judge does not earn promotion.
- **Privacy.** PII scrubbed at ingest; retention TTL on raw trajectories; subject-deletion support; lessons derived from scrubbed text; per-tenant isolation.
- **Not in the request path.** aprntc cannot degrade the live agent's availability or latency.
- **Physical safety for embodied agents.** When the engine acts in the world (§4.9), promotion runs through simulation and teleoperated/shadow trials before any physical canary, and the zero-tolerance safety gate governs collisions, force limits, and human-proximity constraints — not merely text policy. The stakes of a bad promotion are physical, which is the strongest argument for keeping the gate, the human approval, and instant rollback central.

Limitations of the safety case. The judge, though recused and calibrated, remains a model and can err; outcome signals can be gamed by the *environment* (e.g., users who rephrase rather than complain); and auto-promotion, even hard-gated, widens the surface for silent drift. We treat these as reasons to keep the human gate central and the rollback path instant.

9. Limitations

- **Capability ceiling.** Distillation extracts latent competence; it cannot teach the base model new skills. Where the parent is already saturated, expect no gain (§6).
- **Headroom and ground-truth dependence.** Gains require both exploitable parent failures and a reliable outcome signal; domains without crisp outcomes are harder.
- **Judge reliability.** A weak or biased judge limits the ceiling of offline evaluation; we mitigate but do not eliminate this.
- **Prompt-budget pressure.** Accumulating lessons raise context length and latency — the motivation for Phase-2 internalization (§4.8).
- **Capture fidelity for external agents.** For closed, uninstrumented agents, tool-level capture is partial by design; correlation across streams is best-effort.
- **Empirical scope.** Our experiments are in language domains; the multimodal and robotic generalization (§4.9) is architectural and not yet empirically validated here. Embodied outcomes (physical task success, safety) and sim-to-real gating are open work.

10. Conclusion

aprrntc shows that a deployed agent — language, multimodal, or embodied — can improve itself from its own real-world behavior, continuously and safely, without touching model weights. By treating a parent's outcome-labelled trajectories as an experiential substrate and distilling them into a reversible, attributable Playbook — gated by a recused judge and a statistical acceptance test — the system delivers an apprentice that measurably and safely beats its parent *where headroom exists*, and that honestly declines to ship where it does not. The same outcome-anchored supervision extends naturally to multimodal and robotic agents, and to parametric agent SFT once prompt budgets bind. We believe the combination — learn from real outcomes, change only a readable policy artifact, and never ship without a statistical, human-gated, reversible check — is a practical and conservative recipe for self-improving agents.

References

1. P. Abbeel and A. Y. Ng. Apprenticeship Learning via Inverse Reinforcement Learning. *ICML*, 2004.
 2. S. Ross, G. Gordon, and D. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (Dagger). *AISTATS*, 2011.
 3. S. Levine, A. Kumar, G. Tucker, and J. Fu. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *arXiv:2005.01643*, 2020.
 4. P. Christiano et al. Deep Reinforcement Learning from Human Preferences. *NeurIPS*, 2017.
 5. L. Ouyang et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). *NeurIPS*, 2022.
 6. Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022.
 7. P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
 8. E. B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *JASA*, 1927.
 9. L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.
 10. N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*, 2023.
 11. G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*, 2023.
 12. J. S. Park et al. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.
 13. E. Zelikman et al. STaR: Bootstrapping Reasoning with Reasoning. *NeurIPS*, 2022.
 14. T. Anthony, Z. Tian, and D. Barber. Thinking Fast and Slow with Deep Learning and Tree Search (Expert Iteration). *NeurIPS*, 2017.
 15. D. Silver et al. Mastering the Game of Go without Human Knowledge (AlphaZero). *Nature*, 2017.
 16. D. Ha and J. Schmidhuber. World Models. *NeurIPS*, 2018.
 17. D. Hafner et al. Mastering Diverse Domains through World Models (DreamerV3). *arXiv:2301.04104*, 2023.
 18. J. Bruce et al. Genie: Generative Interactive Environments. *ICML*, 2024.
 19. E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
 20. Anthropic. Model Context Protocol (MCP). 2024.
 21. Google. Agent-to-Agent (A2A) Protocol. 2025.
 22. S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.
 23. B. D. Argall, S. Chernova, M. Veloso, and B. Browning. A Survey of Robot Learning from Demonstration. *Robotics and Autonomous Systems*, 2009.
 24. A. Brohan et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv:2307.15818*, 2023.
 25. Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *ICRA*, 2024.
 26. M. J. Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. *CoRL*, 2024.
 27. C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. *RSS*, 2023.
 28. K. Black et al. π 0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv:2410.24164*, 2024.
-

Appendix A Canonical Trajectory Schema (abridged)

Episode { episode_id, task_input, final_output, collector, generation_id, ts_start, partial, turns[] }; Turn { steps[] }; Step { type, tool_name, tool_args, tool_result, duration_ms, reasoning }; plus append-only Labels (outcome, explicit, implicit, judge) and a materialized fused reward. Round-trips every collector's shape; PII scrubbed pre-persistence; subject-deletion and TTL supported.

Appendix B Promotion Gate Defaults

Win-rate $\geq 55\%$; Wilson 95% CI lower bound $> 50\%$ (powered N); loss-rate $< 10\%$; regression failures = 0; new safety violations = 0; promote and rollback verified from the UI. Distillation never sees the held-out gold set.

Appendix C System Facts (reference implementation)

Python core with optional extras per collector ([proxy] [otel] [mcp] [a2a] [byteplus]); FastAPI backend + React/Vite/Tailwind dashboard (review, lineage, trajectories, lessons, try-an-agent; thumbs feedback; Google sign-in; per-tenant isolation); Docker deploy; nightly scheduler with retry/backoff and self-metrics. VikingDB accessed via REST with a hand-rolled Volcengine Signature-V4 signer validated byte-for-byte against the vendor SDK — with the live API as the ultimate oracle.